Aidan D'Alonzo 2/22/2026

# Physics-Informed Neural Nets

A physics-informed neural net (PiNN) is a neural network that incorporates the physical laws of a system, described by general nonlinear partial differential equations, as a regularization agent that provides constraints for admissible solutions when solving supervised learning tasks. Encoding such information into the network allows it to converge to the desired results sooner, and this is helpful because machine learning requires vast amounts of clean training data, which is often sparse or difficult to obtain in practice.

The key concepts within neural net training utilized by PiNNs are automatic differentiation and the back-propagation algorithm. Automatic differentiation refers to the fact that a computer can compute the exact derivative of any function with zero approximation error. Thus, the network is differentiable with respect to its inputs, which allows its partial derivatives to be computed exactly by differentiating through the network itself. The back-propagation algorithm uses this fact, after the input has been fed through the layers and the loss has been calculated, by taking the derivative of that loss and feeding it backwards through the layers to update each parameter's weight accordingly.

PiNNs enrich this process by plugging the output of the network into the PDE of the physical system being studied and combining the physical residual loss with the data loss into a single total loss function, which then gets back-propagated through the layers. A general form for parametrized and nonlinear partial differential equations that can be used is:

$$u_t + \mathcal{N}[u;\lambda] = 0,\ x \in \Omega,\ t \in [0,T], \tag{1}$$

Here, u(t,x) is the hidden solution (some physical quantity that varies with space and time); $\mathcal{N}[\cdot;\lambda]$ is the nonlinear operator that encodes the spatial physics as parametrized by $\boldsymbol{\lambda}$, and $\Omega$ is a subset of $\mathbb{R}^D$.

This can be applied in PiNNs to either solve inference problems or discovery problems. In the first case, there are fixed parameters $\boldsymbol{\lambda}$ and the model predicts u(t,x) - the physics is known and used to find the solution field. In the second case, there is observed data given and the model predicts $\boldsymbol{\lambda}$ - the observations of the system are known and used to find the underlying physical parameters.

I will focus on the former case of inference problems. There are two types of algorithms for computing data-driven solutions to partial differential equations - continuous and discrete time models. In continuous time models, after the spatio-temporal data is entered as input, f(t,x) is defined as $u_t + \mathcal{N}[u]$, the left hand side of equation (1). Thus, it effectively measures how much the physics is violated, as it is meant to measure zero. A deep neural network then approximates u(t,x) (solves for $u_\theta(t,x) \approx u(t,x)$ where $\theta$ are the network weights). Next, using automatic differentiation, the chain rule is applied to compute $u_t$, spatial derivatives, and higher derivatives, which allows the network to build $f_\theta(t,x)$ with the same parameters as $u_\theta(t,x)$. It then evaluates f(t,x) at collocation points, which are just random points in space-time where there might not be

data, but physics still must hold, and forces $u_t + \mathcal{N}[u] \approx 0$. At this point, the program can compute and minimize the mean squared error loss. This is found through the following equation:

$$\mathrm{MSE} = \mathrm{MSE}_u + \mathrm{MSE}_f \quad (2)$$

where $\mathrm{MSE}_u = \frac{1}{Nu}\sum_{i=1}^{Nu} |u(t_u^i, x_u^i) - u^i|^2$ and $\mathrm{MSE}_f = \frac{1}{Nf}\sum_{i=1}^{Nf} |f(t_f^i, x_f^i)|^2$. The $\mathrm{MSE}_u$ corresponds to making the network match with initial and boundary conditions, and thus accounts for the data loss. The $\mathrm{MSE}_f$ corresponds to making the network match with the structure imposed by the PDE, and thus accounts for the physics loss. The network then backpropagates this joint loss and learns a function closer to fitting the observed data and the underlying physics.

Discrete time models operate with similar methodology, but instead of solving the PDEs continuously across all time, they step forward in time using Runge-Kutta. When this is applied, intermediate stages are created inside the timestep, represented by:

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^{q} a_{ij}\mathcal{N}[u^{n+c_j}],\ i = 1,\ldots,q \ (3) \text{ and } u^{n+1} = u^n - \Delta t \sum_{j=1}^{q} b_j\mathcal{N}[u^{n+c_j}] \ (4)$$

Equation (3) shows how each intermediate state equals the previous state minus the timestep multiplied by the weighted combination of physics evaluations. Equation (4) shows how the final update of the next time step is computed using a weighted sum of the stage derivatives. The parameters ($a_{ij}$, $b_j$) are simply the RK coefficients. The discrete time model therefore does not minimize a PDE residual, but minimizes a time-stepping consistency; the network takes x as input and simultaneously outputs u(t,x) at each of the q intermediate RK stages and at the final next timestep. The loss is then computed from the deviation between the NN predictions and the values required by the time-stepping scheme, and gradients of this consistency loss are back-propagated through the network. In this way, the network learns the constraints of the PDE by satisfying the structure of a physics-based time-stepping algorithm, making it a PiNN. Since this method does not need to solve the entire spatio-temporal domain at once, it is preferred for rapidly changing solutions or long time horizon solutions.

An example application of a discrete time model PiNN is solving the Allen-Cahn equation. The Allen-Cahn equation is a fundamental reaction-diffusion PDE that is useful for modeling phase separation in multi-component systems (i.e. the dynamic boundary between water and oil in a container that has been shaken). An example of it given by the referenced research paper is:

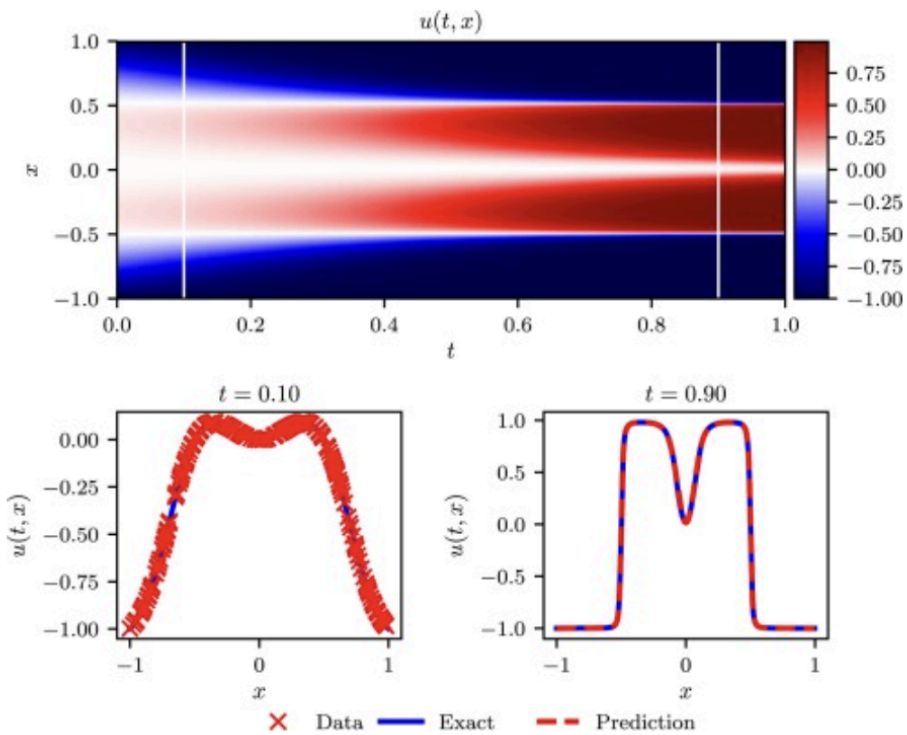
$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad x \in [-1,1], \quad t \in [0,1], \quad (5)$$

with periodic boundary conditions:

$$\begin{aligned} u(0,x) &= x^2\cos(\pi x), \\ u(t,-1) &= u(t,1), \\ u_x(t,-1) &= u_x(t,1). \end{aligned}$$

The $u_{xx}$ term is the diffusion term of the equation and the $5u^3$ - 5u term is the nonlinear reaction term; the former smooths out the interface while the latter sharpens it and drives it toward $\pm 1$ equilibrium phases. Solving the Allen-Cahn equation is classically challenging because the reaction term is highly nonlinear and changes very rapidly near the interface, thus requiring very small timesteps or else the solver overshoots.

A PiNN, however, avoids this problem because the network predicts the solution at all Runge–Kutta stages simultaneously, with shared parameters, and then penalizes any solution that does not satisfy the RK equations. Thus, by having the RK scheme implicit in the network's structure, it is able to take larger timesteps and still be effective without the overshoot worry. In the referenced paper, they tested the PiNN on training data generated by running a conventional spectral method simulation, and it was able to predict the solution from t=0.1 to t=0.9 in one step with a relative error of just 0.00699. The results are displayed graphically below:




The top graph is a heatmap of the true solution u(t,x); it shows the interface narrowing as the competing effects of diffusion and nonlinear reaction drive phase separation. The bottom left graph shows the training snapshot at t=0.1; the blue line is the exact solution while the red X's are the data fed into the network. Lastly the bottom right graph shows the prediction snapshot at t=0.9; the blue line again is the exact solution while the red line is the network's prediction. The incredible overlap of the prediction snapshot shows how the fact that the network couldn't just extrapolate randomly, but had to extrapolate in a way that satisfied the PDE, allowed it to retain high predictive accuracy even while taking a large time step of 0.8.

A future application of PiNNs that I will be exploring is using them to assist in my Wildfire Modeling Capstone, which aims to predict whether a given pixel will be on fire in the next hour using a neural network trained on satellite and meteorological data. Wildfire satellite data is sparse, with limited labeled measurements and significant gaps between passes. A pure data-driven network has no way to constrain its predictions in those gaps, and this could result in it producing predictions that violate physics, like fire teleportation or instantaneous large jumps. A PiNN would be able to avoid that by sampling collocation points and constraining its solution to a governing PDE. I could use the Allen-Cahn equation as an approximation for fire front propagation; the diffusion term would encode how heat spreads outward through unburned fuel,

so the solution has to be spatially smooth. The nonlinear reaction term would drive pixels toward two stable states of either burned (+1) or unburned (-1), so for example if a pixel reaches a burned state it will stay there. This would hopefully prevent the NN from producing predictions that violate basic fire behavior.

**Reference:**

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed Neural Networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045